

Guion para el video — Microservicio de Órdenes de Compra

Duración sugerida: 12-15 minutos. El profesor priorizó que quede bien explicado sobre el tiempo exacto. Puede dividirse en 2 videos si es necesario.

1. Introducción (30 s)

(Muestra el proyecto en el editor)

"Buenas, en este video voy a explicar la segunda fase del proyecto: la implementación de un tercer microservicio, Order.API, encargado de generar órdenes de compra a partir del carrito. A diferencia de Catalog.API y Basket.API, que persisten en PostgreSQL, este microservicio usa MongoDB Atlas como base de datos documental. Voy a explicar la arquitectura del código, y luego una demostración completa: desde agregar productos al carrito hasta consultar la orden generada, cambiarle el estado, descargarla en PDF, y verificar la persistencia directamente en MongoDB Atlas."

2. Arquitectura y código (4-5 min)

(Abre la carpeta src/Order.API en el editor)

"El microservicio sigue el mismo patrón que Catalog.API y Basket.API: ASP.NET Core Minimal API con Carter para el enrutamiento, pero con una separación clara de responsabilidades en cuatro capas."

(Abre Models/PurchaseOrder.cs y Models/OrderItem.cs)

"Primero, el dominio: la clase `PurchaseOrder` representa la orden, con un identificador único, el cliente, la fecha de creación, el estado, la lista de `OrderItem`, y los totales: subtotal, impuesto y total. Cada `OrderItem` guarda el producto, la cantidad, y el precio unitario *congelado* al momento de la compra — no se recalcula después, aunque el precio del catálogo cambie."

(Abre Data/MongoOrderRepository.cs)

"La capa de persistencia usa el driver oficial de MongoDB para .NET. Cada operación pasa por un método `RunAsync` que envuelve las llamadas al driver: si MongoDB no está disponible o hay un timeout, se captura la excepción interna del driver y se traduce a un error genérico. Esto es importante porque el driver por defecto expone detalles internos del clúster en el mensaje de error, y eso viola el requisito de no exponer información sensible al cliente. Aquí también hay un detalle técnico relevante: el driver de MongoDB en su versión 3 requiere declarar explícitamente cómo serializar los `Guid`, cosa que se configura una sola vez al inicio de `Program.cs`."

(Abre Application/OrderService.cs)

"La capa de aplicación es donde vive la lógica de negocio. Al crear una orden, primero se revisa si ya existe una orden con el mismo `Idempotency-Key` para ese cliente — si existe, se devuelve esa misma orden en lugar de crear una nueva. Esto implementa idempotencia: si el cliente reintenta la misma solicitud, por ejemplo por un error de red, no se genera una compra duplicada. Después, se obtiene el carrito llamando por HTTP a Basket.API, se valida que no esté vacío, y por cada producto se valida que exista consultando a Catalog.API. Si algo falla en cualquiera de estas validaciones, se lanza una excepción que se traduce a `400 Bad Request`. Aquí también está la máquina de estados: un diccionario define las transiciones válidas — de

Pending se puede pasar a Confirmed o a Cancelled , pero desde esos dos estados finales no se puede transicionar a ningún otro."

(Abre Endpoints/OrdersEndpoint.cs)

"Finalmente, los endpoints: POST /api/orders para crear, GET /api/orders para listar todas, GET /api/orders/{id} para consultar una específica, GET /api/orders/customer/{customerId} para las de un cliente, y PATCH /api/orders/{id}/status para cambiar el estado. Todo documentado con Swagger."

(Abre frontend/src/App.vue)

"Del lado del frontend, en Vue 3, agregué un carrito en un panel deslizable activado por un ícono, para no saturar la pantalla principal. Al confirmar la compra, se abre el detalle de la orden en una pestaña nueva — usando parámetros de URL para simular navegación sin agregar un router completo—, y desde ahí se puede cambiar el estado de la orden y descargar un comprobante en PDF generado en el navegador con la librería jsPDF. También agregué una vista de 'Pedidos' que lista todas las órdenes, con filtro por cliente y paginación."

3. Demo completa en producción (7-8 min)

(Abre la URL real de Netlify)

"Ahora una demostración completa, todo contra el entorno de producción: frontend en Netlify, y los tres microservicios —Catalog, Basket y Order— desplegados en Render, cada uno con su propia base de datos en la nube."

(Agrega 1-2 productos al carrito con el ícono)

"Agrego productos al carrito desde el catálogo."

(Abre el carrito, clic en "Realizar compra")

"Al realizar la compra, el frontend llama a POST /api/orders con el customerId , el basketId , y un Idempotency-Key generado automáticamente. La orden se abre en una pestaña nueva, mostrando el identificador único, la fecha, la hora, los productos con sus cantidades y precios, y el estado inicial: Pending ."

(Ve a MongoDB Atlas → Browse Collections → base OrdersDb, colección orders)

"Para comprobar que la persistencia es real y no solo en memoria, entro a MongoDB Atlas y busco el documento con el mismo identificador que acabamos de ver — aquí está, con exactamente los mismos datos: cliente, items, subtotal, impuesto, total y estado."

(Vuelve a la pestaña de la orden, clic en "Confirmar orden")

"Cambio el estado de la orden de Pending a Confirmed , usando el endpoint PATCH /api/orders/{id}/status , que valida que la transición sea permitida."

(Clic en "Descargar PDF")

"Y descargo el comprobante en PDF, generado del lado del cliente con fecha, hora, productos, cantidades, precios y total."

(Abre la vista de Pedidos desde el botón del catálogo)

"En la vista de Pedidos se listan todas las órdenes generadas, con paginación, y puedo filtrar por cliente específico para ver solo sus compras."

(Abre Swagger de Order.API en otra pestaña: /swagger/index.html)

"Para cerrar, algunas pruebas puntuales desde Swagger. Primero, un carrito vacío."

(Prueba POST /api/orders con un cliente sin carrito)

"Responde `400 Bad Request` , como exige la regla de negocio."

(Prueba POST /api/orders dos veces con el mismo Idempotency-Key)

"Ahora, reenvío la misma solicitud con el mismo `Idempotency-Key` : en vez de crear una segunda orden, el sistema devuelve la orden que ya existía — esto demuestra la idempotencia."

(Prueba PATCH para pasar una orden Cancelled a Confirmed)

"Y una transición de estado inválida, de `Cancelled` a `Confirmed` : el sistema la rechaza con `400 Bad Request` , protegiendo la integridad del ciclo de vida de la orden."

4. Cierre (30 s)

"En resumen: Order.API es un microservicio independiente con Minimal API, persistencia en MongoDB Atlas, idempotencia mediante `Idempotency-Key` , control de estados con transiciones validadas, e integración HTTP con los microservicios existentes de catálogo y carrito — todo publicado en la nube y funcionando en conjunto con el frontend. Gracias."

Checklist antes de grabar

- ☐ Los 3 servicios en Render respondiendo (/products, /health, /swagger/index.html de order-api).
- ☐ MongoDB Atlas con el cluster activo (no pausado por inactividad — revisa antes de grabar).
- ☐ Un carrito de prueba ya cargado en Netlify para no perder tiempo en vivo.
- ☐ Swagger de Order.API abierto en una pestaña, listo para las pruebas de basket vacío / idempotencia / transición inválida.
- ☐ Contraseñas/connection strings tapadas si el video se comparte públicamente.